Object Oriented Programming

Prof. Dr. Muhammad Usman
Department of Computer Science
UET Mardan

Email: usman@uetmardan.edu.pk

CONTROLLING ACCESS TO MEMBERS

Class member access

- Default **`private`**
- Explicitly set to **`private`**, **`public`**, **`protected`**

`struct` member access

- Default **`public`**
- Explicitly set to **`private`**, **`public`**, **`protected`**

Access to class's **`private`** data

- Controlled with access functions (accessor methods)
  - Get function
    - Read **`private`** data
  - Set function
    - Modify **`private`** data

ACCESS FUNCTIONS AND UTILITY FUNCTIONS

Access functions

- **`public`**
- Read/display data
- Predicate functions
  - Check conditions

Utility functions (helper functions)

- **`private`**
- Support operation of **`public`** member functions
- Not intended for direct client use

SALESP.H (1 OF 1)

```
1   // salesp.h
2   // SalesPerson class definition.
3   // Member functions defined in salesp.cpp.
4   #ifndef SALESP_H
5   #define SALESP_H
6
7   class SalesPerson {
8
9   public:
10     SalesPerson();                  // constructor
11     void getSalesFromUser();        // input sales from keyboard
12     void setSales( int, double ); // set sales for a month
13     void printAnnualSales();        // sum                    sales
14
15  private:
16     double totalAnnualSales();     // utility function
17     double sales[ 12 ];            // 12 monthly sales figures
18
19  }; // end class SalesPerson
20
21  #endif
```

Set access function performs validity checks.

`private` utility function.

SALESP.CPP (1 OF 3)

```
1   // salesp.cpp
2   // Member functions for class SalesPerson.
3   #include <iostream>
4
5   using std::cout;
6   using std::cin;
7   using std::endl;
8   using std::fixed;
9
10  #include <iomanip>
11
12  using std::setprecision;
13
14  // include SalesPerson class definition from salesp.h
15  #include "salesp.h"
16
17  // initialize elements of array sales to 0.0
18  SalesPerson::SalesPerson()
19  {
20     for ( int i = 0; i < 12; i++ )
21        sales[ i ] = 0.0;
22
23  } // end SalesPerson constructor
24
```

SALESP.CPP (2 OF 3)

```
25  // get 12 sales figures from the user at the keyboard
26  void SalesPerson::getSalesFromUser()
27  {
28     double salesFigure;
29
30     for ( int i = 1; i <= 12; i++ ) {
31        cout << "Enter sales amount for month " << i << ": ";
32        cin >> salesFigure;
33        setSales( i, salesFigure );
34
35     } // end for
36
37  } // end function getSalesFromUser
38
39  // set one of the 12 monthly sales figures; function subtracts
40  // one from month value for proper subscript in sales array
41  void SalesPerson::setSales( int month, double amount )
42  {
43     // test for valid month and amount values
44     if ( month >= 1 && month <= 12 && amount > 0 )
45        sales[ month - 1 ] = amount; // adjust for subscripts 0-11
46
47     else // invalid month or amount value
48        cout << "Invalid month or sales figure" << endl;
```

Set access function performs validity checks.

SALESP.CPP (3 OF 3)

```
49
50 } // end function setSales
51
52 // print total annual sales (with help of utility function)
53 void SalesPerson::printAnnualSales()
54 {
55    cout << setprecision( 2 ) << fixed
56         << "\nThe total annual sales are: $"
57         << totalAnnualSales() << endl; // call utility function
58
59 } // end function printAnnualSales
60
61 // private utility function to total annual sales
62 double SalesPerson::totalAnnualSales()
63 {
64    double total = 0.0;           // initialize total
65
66    for ( int i = 0; i < 12; i++ )  // summarize sales results
67       total += sales[ i ];
68
69    return total;
70
71 } // end function totalAnnualSales
```

`private` utility function to help function **`printAnnualSales`**; encapsulates logic of manipulating **`sales`** array.

MAINPROG.CPP

```
1   // mainprog.cpp
2   // Demonstrating a utility function.
3   // Compile this program with salesp.cpp
4
5   // include SalesPerson class definition from salesp.h
6   #include "salesp.h"
7
8   int main()
9   {
10     SalesPerson s;          // create
11
12     s.getSalesFromUser();  // note simple sequential code;
13     s.printAnnualSales();  // no control structures in main
14
15     return 0;
16
17  } // end main
```

Simple sequence of member function calls; logic encapsulated in member functions.

```
Enter sales amount for month 1: 5314.76

Enter sales amount for month 2: 4292.38

Enter sales amount for month 3: 4589.83

Enter sales amount for month 4: 5534.03

Enter sales amount for month 5: 4376.34

Enter sales amount for month 6: 5698.45

Enter sales amount for month 7: 4439.22

Enter sales amount for month 8: 5893.57

Enter sales amount for month 9: 4909.67

Enter sales amount for month 10: 5123.45

Enter sales amount for month 11: 4024.97

Enter sales amount for month 12: 5923.92


The total annual sales are: $60120.59
```

USING *SET* AND *GET* FUNCTIONS

Set functions

- Perform validity checks before modifying **`private`** data
- Notify if invalid values
- Indicate with return values

Get functions

- "Query" functions
- Control format of data returned

TIME3.H (1 OF 2)

```
1   // time3.h
2   // Declaration of class Time.
3   // Member functions defined in time3.cpp
4
5   // prevent multiple inclusions of header file
6   #ifndef TIME3_H
7   #define TIME3_H
8
9   class Time {
10
11  public:
12     Time( int = 0, int = 0, int = 0 );  // defau
13
14     // set functions
15     void setTime( int, int, int );  // set hour, minute, second
16     void setHour( int );    // set hour
17     void setMinute( int ); // set minute
18     void setSecond( int ); // set second
19
20     // get functions
21     int getHour();          // return hour
22     int getMinute();        // return minute
23     int getSecond();        // return second
24
```

Set functions.

Get functions.

TIME3.H (2 OF 2)

```
25      void printUniversal(); // output universal-time format
26      void printStandard();  // output standard-time format
27
28  private:
29      int hour;             // 0 - 23 (24-hour clock format)
30      int minute;           // 0 - 59
31      int second;           // 0 - 59
32
33  }; // end clas Time
34
35  #endif
```

TIME3.CPP (1 OF 4)

```
1  // time3.cpp
2  // Member-function definitions for Time class.
3  #include <iostream>
4
5  using std::cout;
6
7  #include <iomanip>
8
9  using std::setfill;
10 using std::setw;
11
12 // include definition of class Time from time3.h
13 #include "time3.h"
14
15 // constructor function to initialize private data;
16 // calls member function setTime to set variables;
17 // default values are 0 (see class definition)
18 Time::Time( int hr, int min, int sec )
19 {
20    setTime( hr, min, sec );
21
22 } // end Time constructor
23
```

TIME3.CPP (2 OF 4)

```
24 // set hour, minute and second values
25 void Time::setTime( int h, int m, int s )
26 {
27    setHour( h );
28    setMinute( m );
29    setSecond( s );
30
31 } // end function setTime
32
33 // set hour value
34 void Time::setHour( int h )
35 {
36    hour = ( h >= 0 && h < 24 ) ? h : 0;
37
38 } // end function setHour
39
40 // set minute value
41 void Time::setMinute( int m )
42 {
43    minute = ( m >= 0 && m < 60 ) ? m : 0;
44
45 } // end function setMinute
46
```

Call set functions to perform validity checking.

Set functions perform validity checks before modifying data.

TIME3.CPP (3 OF 4)

```
47  // set second value
48  void Time::setSecond( int s )
49  {
50    second = ( s >= 0 && s < 60 ) ? s : 0;
51
52  } // end function setSecond
53
54  // return hour value
55  int Time::getHour()
56  {
57    return hour;
58
59  } // end function getHour
60
61  // return minute value
62  int Time::getMinute()
63  {
64    return minute;
65
66  } // end function getMinute
67
```

Set function performs validity checks before modifying data.

Get functions allow client to read data.

TIME3.CPP (4 OF 4)

```
68  // return second value
69  int Time::getSecond()
70  {
71     return second;
72
73  } // end function getSecond
74
75  // print Time in universal format
76  void Time::printUniversal()
77  {
78     cout << setfill( '0' ) << setw( 2 ) << hour << ":"
79          << setw( 2 ) << minute << ":"
80          << setw( 2 ) << second;
81
82  } // end function printUniversal
83
84  // print Time in standard format
85  void Time::printStandard()
86  {
87     cout << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )
88          << ":" << setfill( '0' ) << setw( 2 ) << minute
89          << ":" << setw( 2 ) << second
90          << ( hour < 12 ? " AM" : " PM" );
91
92  } // end function printStandard
```

Get function allows client to read data.

MAINPROG.CPP (1 OF 3)

```
1   // mainprog.cpp
2   // Demonstrating the Time class set and get functions
3   #include <iostream>
4
5   using std::cout;
6   using std::endl;
7
8   // include definition of class Time from time3.h
9   #include "time3.h"
10
11  void incrementMinutes( Time &, const int );  // prototype
12
13  int main()
14  {
15     Time t;              // create Time object
16
17     // set time using individual set functions
18     t.setHour( 17 );     // set hour to valid value
19     t.setMinute( 34 );   // set minute to valid value
20     t.setSecond( 25 );   // set second to valid value
21
```

Invoke set functions to set valid values.

MAINPROG.CPP (2 OF 3)

```
22    // use get functions to obtain hour, minute and second
23    cout << "Result of setting all valid values:\n"
24         << "  Hour: " << t.getHour()
25         << "  Minute: " << t.getMinute()
26         << "  Second: " << t.getSecond();
27
28    // set time using individual set functions
29    t.setHour( 234 );    // invalid hour set to 0
30    t.setMinute( 43 );   // set minute to valid value
31    t.setSecond( 6373 ); // invalid second set to 0
32
33    // display hour, minute and second after setting
34    // invalid hour and second values
35    cout << "\n\nResult of attempting to set invalid hour and"
36         << " second:\n  Hour: " << t.getHour()
37         << "  Minute: " << t.getMinute()
38         << "  Second: " << t.getSecond() << "\n\n";
39
40    t.setTime( 11, 58, 0 );    // set time
41    incrementMinutes( t, 3 );  // increment t's minute by 3
42
43    return 0;
44
45 } // end main
46
```

Attempt to set invalid values using set functions.

Invalid values result in setting data members to **0**.

Modify data members using function **`setTime`**.

MAINPROG.CPP (2 OF 3)

```
47  // add specified number of minutes to a Time object
48  void incrementMinutes( Time &tt, const int count )
49  {
50     cout << "Incrementing minute " << count
51          <
52     tt.pr
53
54     for ( int i = 0; i < count; i++ ) {
55        tt.setMinute( ( tt.getMinute() + 1 ) % 60 );
56
57        if ( tt.getMinute() == 0 )
58           tt.setHour( ( tt.getHour() + 1 ) % 24);
59
60        cout << "\nminute + 1: ";
61        tt.printStandard();
62
63     } // end for
64
65     cout << endl;
66
67  } // end function incrementMinutes
```

Using get functions to read data and set functions to modify data.

```
Result of setting all valid values:

  Hour: 17  Minute: 34  Second: 25


Result of attempting to set invalid hour and second:

  Hour: 0  Minute: 43  Second: 0


Incrementing minute 3 times:

Start time: 11:58:00 AM

minute + 1: 11:59:00 AM

minute + 1: 12:00:00 PM

minute + 1: 12:01:00 PM
```

Attempting to set data members with invalid values results in error message and members set to 0.

COMPOSITION: OBJECTS AS MEMBERS OF CLASSES

- Composition is a concept where a **class contains objects of other classes as its members**. It represents a **"has-a"** relationship between classes.
  - Car HAS-A Engine
  - House HAS-A Room
  - Student HAS-A Address
  - Person HAS-A BirthDate

This is different from **inheritance** which is an **"is-a"** relationship:

- Dog IS-A Animal ← inheritance
- Car HAS-A Engine ← composition

COMPOSITION: OBJECTS AS MEMBERS OF CLASSES

- Constructor of class having object of other classes runs first, but constructors of other classes complete first
- Member objects are constructed in order in which they are declared and not in order of the constructors member initializer list
- Member objects are constructed before enclosing class objects (host objects)
- Constructor of the composition class must use member initializer syntax for all data member initialization

COMPOSITION - EXAMPLE

```
// ------ interface.h ------
#ifndef INTERFACE_H
#define INTERFACE_H

#include <iostream>
using namespace std;

// PART class — used inside another class
class Engine {
private:
    int   horsepower;
    float displacement;
public:
    Engine() {
        horsepower   = 0;
        displacement = 0.0;
        cout << "Engine Default Constructor\n";
    }
    Engine(int hp, float disp) {
        horsepower   = hp;
        displacement = disp;
        cout << "Engine Parameterized Constructor\n";
    }
    void showEngine() {
        cout << "Engine: " << horsepower  << "hp, "
                           << displacement << "L\n";
    }
};
```

```
// WHOLE class — contains Engine object
class Car {
private:
    string brand;
    int    year;
    Engine engine;       // ← composition — Car HAS-A Engine
public:
    Car() {
        brand  = "Unknown";
        year   = 0;
        cout << "Car Default Constructor\n";
    }
    Car(string b, int y, Engine e) {
        brand  = b;
        year   = y;
        engine = e;
        cout << "Car Parameterized Constructor\n";
    }
    void showCar() {
        cout << "Brand: " << brand << endl;
        cout << "Year:  " << year  << endl;
        engine.showEngine();     // calling member object's function
    }
};
#endif
```

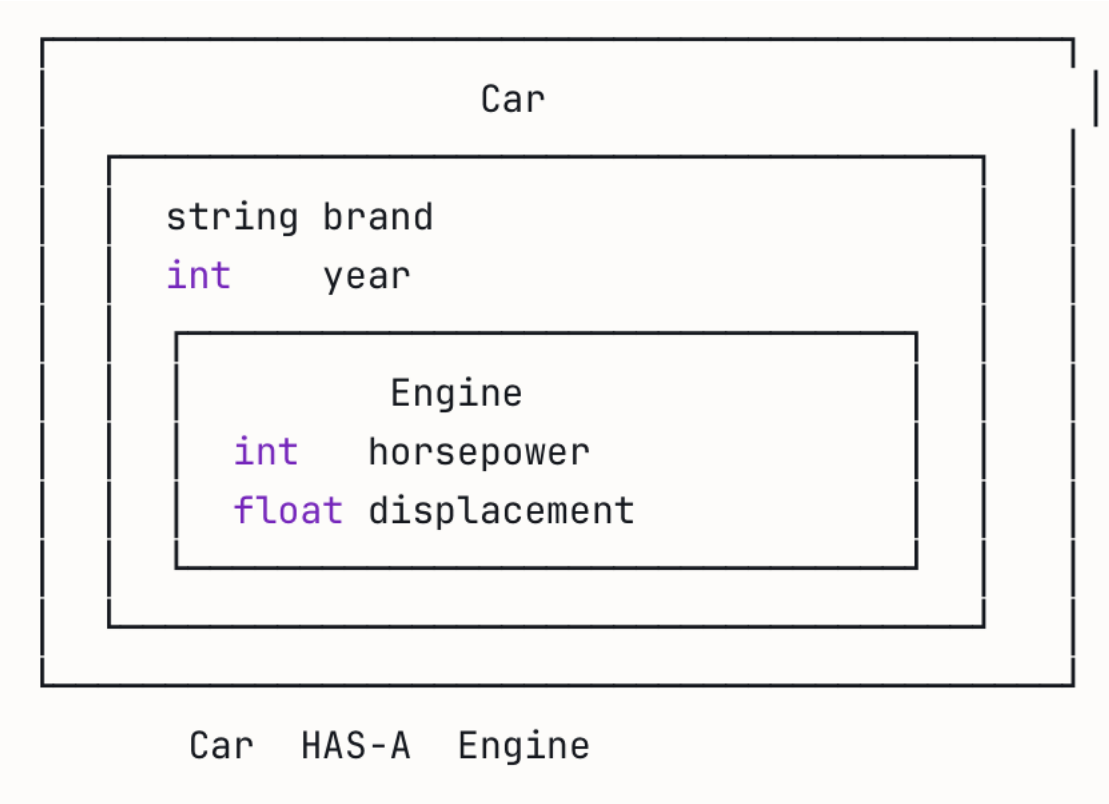
```
// ------ mainprog.cpp ------
#include "interface.h"

int main() {
    // create Engine object first
    Engine e1(200, 2.5);

    // create Car object with Engine
    Car c1("Toyota", 2024, e1);

    cout << "\n--- Car Details ---\n";
    c1.showCar();

    return 0;
}
```



Output:

```
Engine Parameterized Constructor
Engine Default Constructor      ← Car's default engine member initialized
Car Parameterized Constructor

--- Car Details ---
Brand: Toyota
Year:  2024
Engine: 200hp, 2.5L
```

- Composition builds **complex classes from simpler ones** by including objects as members.
- Member object constructors are always called **before** the containing class constructor, and member objects are destroyed **after** the containing object's destructor runs.

DATE1.H (1 OF 1)

```
1   // date1.h
2   // Date class definition.
3   // Member functions defined in date1.cpp
4   #ifndef DATE1_H
5   #define DATE1_H
6
7   class Date {
8
9   public:
10     static const int monthsPerYear = 12; // number of months in a year
       Date( int = 1, int = 1, int = 1900 ); // default constructor
11     void print() const;  // print date in month/day/year format
12     ~Date();  // provided to confirm destruction order
13
14  private:
15     int month;  // 1-12 (January-December)
16     int day;    // 1-31 based on month
17     int year;   // any year
18
19     // utility function to test proper day for month and year
20     int checkDay( int ) const;
21
22  }; // end class Date
23
24  #endif
```

Note no constructor with parameter of type **`Date`**. Recall compiler provides default copy constructor.

Declared and Defined Both???

ANOTHER EXAMPLE

DATE1.CPP (1 OF 3)

```
1   // date1.cpp
2   // Member-function definitions for class Date.
3   #include <iostream>
4
5   using std::cout;
6   using std::endl;
7
8   // include Date class definition from date1.h
9   #include "date1.h"
10
11  // constructor confirms proper value for month; calls
12  // utility function checkDay to confirm proper value for day
13  Date::Date( int mn, int dy, int yr )
14  {
15     if ( mn > 0 && mn <= 12 )  // validate the month
16        month = mn;
17
18     else {                     // invalid month set to 1
19        month = 1;
          cout << "Month " << mn << " invalid. Set to month 1.\n";
21     }
20
23     year = yr;                 // should validate yr
24     day = checkDay( dy );      // validate the day
25
```

DATE1.CPP (2 OF 3)

```
20. // output Date object to show when its constructor is called
21.     cout << "Date object constructor for date ";
22.     print();
23.     cout << endl;
24. } // end Date constructor

26. // print Date object in form month/day/year
27. void Date::print() const
28. {
29.     cout << month << '/' << day << '/' << year;
30. } // end function print
31.// output Date object to show when its destructor is called
32.Date::~Date() {
33.  cout << "Date object destructor for date ";
34.print();
35.cout << endl;
36.} // end ~Date destructor
```

DATE1.CPP (3 OF 3)

```
// utility function to confirm proper day value based on month and year; handles leap years, too
int Date::checkDay( int testDay ) const
{

static const int daysPerMonth[ monthsPerYear + 1 ] =
{ 0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

// determine whether testDay is valid for specified month

if ( testDay > 0 && testDay <= daysPerMonth[ month ] )

  return testDay;

// February 29 check for leap year

if ( month == 2 && testDay == 29 && ( year % 400 == 0 || ( year % 4 == 0 && year % 100 != 0 ) ) )
  return testDay;

else

 cout <<  "Invalid day for current month and year"; // end function checkDay
```

EMPLOYEE1.H (1 OF 2)

```
1   // employee1.h
2   // Employee class definition.
3   // Member functions defined in employee1.cpp.
4   #ifndef EMPLOYEE1_H
5   #define EMPLOYEE1_H
6
7   // include Date class definition from date1.h
8   #include "date1.h"
9
10  class Employee {
11
12  public:
13     Employee(const char *, const char *, const Date &, const Date & );
15
16     void print() const;
17     ~Employee();  // provided to confirm destruction order
18
19  private:
20     char firstName[ 25 ];
21     char lastName[ 25 ];
22     const Date birthDate;  // composition: member object
23     const Date hireDate;   // composition: member object
24  }; // end class Employee
25  #endif
```

Using composition; **`Employee`** object contains **`Date`** objects as data members.

EMPLOYEE1.CPP(1 OF 3)

```
1  // employee1.cpp
2  // Member-function definitions for class Employee.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include <cstring>      // strcpy and strlen prototypes
9
10 #include "employee1.h"  // Employee class definition
11 #include "date1.h"      // Date class definition
12
```

EMPLOYEE1.CPP(2 OF 3)

```
13 // constructor uses member initializer list to pass initializer
14 // values to constructors of member objects birthDate and
15 // hireDate [Note: This invokes the so-called "default copy
16 // constructor" which the C++ compiler provides implicitly.]
17 Employee::Employee( const char *first, const char *last,
18    const Date &dateOfBirth, const Date &dateOfHire )
19    : birthDate( dateOfBirth ),  // initialize birthDate
20      hireDate( dateOfHire )     // initialize hireDate
21 {
22    // copy first into firstName and be sure that it fits
23    int length = strlen( first );
24    length = ( length < 25 ? length : 24 );
25    strncpy( firstName, first, length );
26    firstName[ length ] = '\0';
27
28    // copy last into lastName and be sure that it fits
29    length = strlen( last );
30    length = ( length < 25 ? length : 24 );
31    strncpy( lastName, last, length );
32    lastName[ length ] = '\0';
33
34    // output Employee object to show when constructor is called
35    cout << "Employee object constructor: "
36         << firstName << ' ' << lastName << endl;
37
```

Member initializer syntax to initialize **`Date`** data members **`birthDate`** and **`hireDate`**; compiler uses default copy constructor.

Output to show timing of constructors.

EMPLOYEE1.CPP(3 OF 3)

```
38  } // end Employee constructor
39
40  // print Employee object
41  void Employee::print() const
42  {
43     cout << lastName << ", " << firstName << "\nHired: ";
44     hireDate.print();
45     cout << "  Birth date: ";
46     birthDate.print();
47     cout << endl;
48
49  } // end function print
50
51  // output Employee object to show whe
52  Employee::~Employee()
53  {
54     cout << "Employee object destructor: "
55          << lastName << ", " << firstName << endl;
56
57  } // end destructor ~Employee
```

Output to show timing of destructors.

MAINPROG.CPP

```
1   // mainprog.cpp
2   // Demonstrating composition--an object with member objects.
3   #include <iostream>
4
5   using std::cout;
6   using std::endl;
7
8   #include "employee1.h"  // Employee class definition
9
10  int main()
11  {
12     Date birth( 7, 24, 1949 );
13     Date hire( 3, 12, 1988 );
14     Employee manager( "Bob", "Jones", birth, hire );
15
16     cout << '\n';
17     manager.print();
18
19     cout << "\nTest Date constructor with invalid values:\n";
20     Date lastDayOff( 14, 35, 1994 );  // invalid month and day
21     cout << endl;
22
23     return 0;
24
25  } // end main
```

```
Date object constructor for date 7/24/1949
Date object constructor for date 3/12/1988
Employee object constructor: Bob Jones


Jones, Bob
Hired: 3/12/1988  Birth date: 7/24/1949


Test Date constructor with invalid values:
Month 14 invalid. Set to month 1.
Day 35 invalid. Set to day 1.
Date object constructor for date 1/1/1994


Date object destructor for date 1/1/1994
Employee object destructor: Jones, Bob
Date object destructor for date 3/12/1988
Date object destructor for date 7/24/1949
Date object destructor for date 3/12/1988
Date object destructor for date 7/24/1949
```

Note two additional **`Date`** objects constructed; no output since default copy constructor used.

Destructor for host object **`manager`** runs before

Destructor for **`Employee`**'s

Destructor for **`Employee`**'s

Destructor for **`Date`** object **`hire`**

Destructor for **`Date`** object **`birth`**.